Spreading the Load using Consistent Hashing: A Preliminary Report

Garret Swart

Abstract—**Consistent hashing can be used to assign objects to nodes in a distributed system. It has been used by several distributed systems including Chord, Pastry, and Tornado because of its efficient handling of node failure and repair. In this paper we analyze how well consistent hashing does at evenly distributing objects among the nodes in the system. We also extend current consistent hashing algorithms to allow for dynamic load balancing while retaining the good properties of consistent hashing. Finally we analyze our extensions using both probabilistic analysis and simulations. The algorithms derived appear to achieve much better load balancing.**

Index Terms— **Distributed information systems, Combinatorial mathematics, Distributed algorithms, Load modeling**

I. Introduction

Consistent hashing [1] is a technique used in building and maintaining a distributed hash table where each node stores a set of objects that hash to the node. The beauty of consistent hashing is that when a node fails or is repaired only a small number of objects need to be relocated to a small number of other nodes. This is important because in large distributed systems failures and repairs are part of the system's normal behaviour, with the rate of failures and repairs growing linearly with the size of the system. In such an environment if the cost of handling a failure or repair is too large then system can spend most of its time reconfiguring and no time actually getting work done.

Using traditional distributed hashing as implemented in Oracle Parallel Server or in OpenVMS [2], each node in an n node system is assigned an index between 0 and $n-1$ and each object is hashed to a number between 0 and $n-1$ and is assigned to the corresponding node. If a node fails or is repaired, the nodes are renumbered and each of the objects are rehashed causing massive overheads as objects are moved to the correct node. However this approach is simple and works well as long as there are small numbers of machines in the cluster.

Consistent hashing can be thought of as hashing the object keys and the node IDs to points along the circumference of a circle. Each object is assigned to the node whose hash immediately follows it moving clockwise around the circle.

Using this approach, when a node is added, it splits the load with one existing node, while if a node fails, its load falls onto its adjacent node. If we want the objects maintained reliably we assign each object to a set of r nodes. The object will be available if a large enough subset of the r nodes storing a replica of the object is available; the size of the subset needed depends on the details of the replication algorithm being used. The replication algorithm and the number r are chosen so that the probability that the object is available is large enough for the intended purpose. The original consistent hashing paper [1] suggests storing the r copies of the object on the r nodes hashed to positions following the hash position of the object.

Consistent hashing has now been implemented in several different systems including Chord [3], Pastry [4], and Tornado [5]. A potential problem with consistent hashing is that the resulting system may be prone to overload, that is, to having too many objects assigned to a single node. Overload in such a system can be fatal: if even one node is overloaded and it cannot serve the objects assigned to it then it may fail. When a node fails, the load from that node will pass on the node adjacent node, perhaps causing the load on that node to reach a level where it too becomes overloaded and fails. This kind of cascading failure catastrophe is something that must be avoided.

In this paper we analyse the probability of overload in consistent hashing and present several modifications to the replica assignment algorithm that reduces the probability of overload at significantly less cost than existing solutions.

II. Overload Policies

Instead of continuing to accept load until failure, an alternative reaction to excess load is to have a node refuse to accept additional load when it is nearing its overload threshold. However this can cause the object to become lost because objects cannot be placed on any node, they must be placed where the hashing algorithm specifies that they should be placed if a search is to find the object.

Note that overload of this type may not be a severe problem for systems using consistent hashing where the overhead of hosting an object is very small. For example, the objects in the distributed hash table may not be used to distribute load bearing objects themselves, they may instead just contain pointers to the location of those objects. This is the case in the Pier database system [6] where the objects being distributed are distributed indexes rather than the actual data. It is also used in Oracle Parallel Server distributed block cache, where

Manuscript received April 2, 2004, revised May 26, 2004.

Garret Swart is a College Lecturer at University College Cork in Cork, Ireland; e-mail: g.swart@cs.ucc.ie.

the hashed node is only used to store the lock information for the block rather than the block itself. This approach has the disadvantage that finding the data costs an extra level of indirection and begs the question of how the load bearing objects themselves should be distributed.

However for other systems, the objects themselves represent a significant commitment of resources, for example in CFS [7] each object could represent an 8K disk block that needs to be stored on disk and on which read and write operations have to be performed. In such a cooperative file system the user may have specified a hard limit on the amount of resources to be used by the file system on his machine. We have to make sure that the probability that any user's system will get more load than allowed is near zero.

The Chord paper [3] discusses overload briefly in its section V.B. and presented a partial solution to the problem called *virtual nodes*. The idea is that instead of hashing each node to a single point on the unit interval, we hash it to v different points on the interval. Each point that a node hashes to is called a virtual node. This technique can be used for load balancing: by increasing the number of virtual nodes per real node the probability of overload is reduced because the variance in the number of objects that are assigned to each node is decreased. However virtual nodes also have an overhead on the real nodes because the node must maintain itself in v places along the circle. This entails talking to the s virtual nodes around it in each of the v places it appears. The number s is chosen so that $s \geq r$ and so that the probability of s consecutive nodes becoming unavailable in a short period of time anywhere in the circle is very small.

Using virtual nodes also reduces the probability of a cascade failure catastrophe like that mentioned in the previous section, the failure of a node due to overload does not send all of its load to a single adjacent node, but instead the load is distributed among the v nodes adjacent to the node's virtual nodes.

III. An Analysis of Chord Load

In what follows use the following definitions:

m: The number of objects being stored in the system.

n: The total number of nodes in the system.

r: The level of replication: Each object is stored on at least r nodes. When one of those nodes fails, a replacement is sought.

v: The number of virtual nodes assigned to each real node. Each node is assigned to v hash values.

s: The *spread* of each virtual node, that is the number of virtual nodes in a row around the circle that each virtual node must be aware of so that the network can be maintained in the case of failures. Note that $s \geq r$ as all the replicas must be in the spread of the others so that they can be monitored.

For now we assume that each node is identical and that each node has the same number of virtual nodes assigned to it. The case where nodes have significant variations in capacity is discussed briefly at the end of the paper.

The expected number of objects that is stored on a node can easily be seen to be rm/n, this is because each object is stored r times and the load of rm objects is spread over n nodes. The question that we would like to answer is: What is the number of objects we should expect to see stored on the most heavily loaded node on the system?

Since we don't know which node will be the most heavily loaded, all nodes must be ready to provide resources sufficient for this maximum number of objects. The larger the expected maximum is than the expected number of objects per node, the greater the amount of wasted resources there will be in the system. In particular we want to know how much of our available resources we need to leave idle to allow for these few overloaded nodes. Conversely, if we decide not to allocate enough resources, then the larger this expected maximum is, the greater the probability of overload.

The first result of this paper is to compute the probability that the load on a particular node has a given value. The second result is to compute the probability that the node with the maximum load stores a given number of objects. The final result is to compute the standard deviation of the object loads. These results should be compared with theorem 4.1 and 4.5 of [1].

A. Probability Distribution of Node Load

In all of the analysis that follows we will assume that the hash functions are perfect, that is that they map the nodes and objects uniformly and independently at random around the unit circle.

We will first determine the probability function that any particular node has k objects assigned to it and then use that to compute the cumulative probability F that a node has k or fewer objects assigned to it.

Let us first look at the case where $v = 1$. If we choose an arbitrary node, the probability that it has exactly k objects assigned to it is:

$$\Pr(Load = k) = \frac{\binom{k + rv - 1}{k}\binom{nv + m - rv - k - 1}{m - k}}{\binom{nv + m - 1}{m}} \qquad (1)$$

This is the ratio of the number of ways of assigning exactly k of the m objects to load one node divided by the total number of ways of assigning m objects to n nodes, in any fashion.

To determine the number of ways of assigning objects to nodes, think of the set of n nodes and m objects as undifferentiated items placed around a circle and then let us look at the ways that we can label these items as either nodes or objects. Without loss of generality we choose one of the items on the circle to be a node and look at all the ways that that node can have a load of k objects. For the chosen node to have a load of k, the $r + k - 1$ items that follow the chosen node must contain exactly $r - 1$ nodes at any position, and the item following those items must itself be a node. The rest of the items may be assigned arbitrarily as long as $m - k$ of the

remaining items are chosen as objects.

The first term is the number of ways of assigning k of the items to be objects following or chosen node, while the second term is the number of ways of assigning the rest of the items to represent the remaining objects and items. The denominator is the number of ways of choosing m of the remaining items to be objects in an arbitrary way.

We can derive the simpler case of $v=1$ and $r=1$ by considering the problem in the continuous domain. Consider a unit perimeter circle split into n segments by n points uniformly chosen on the circle. Without loss of generality, pick one of the points and use it to split the circle into a unit interval with the remaining $n-1$ points splitting the interval into n segments. The length of the first segment is the minimum of the $n-1$ values uniformly chosen on the unit interval. By symmetry the probability density of the arc length is then same as that given by as the minimum of $n-1$ uniform random variables on the range 0 to 1, that is, the probability that $n-2$ of the intervals have size larger than p times the number of ways of choosing which interval is to be the longest. This is:

$$(n-1)(1-p)^{n-2}. \tag{2}$$

Conditioning on the random interval length being equal to a particular value p, the number of objects in the interval is given by a binomial distribution with m trials with probability p, that is a probability density function of:

$$\binom{m}{k} p^k (1-p)^{m-k}. \tag{3}$$

To get the resulting unconditioned probability distribution we can convolve the two distributions in (2) and (3) giving us:

$$\begin{aligned}
&\Pr(Load_{v=1\wedge r=1} = k) \\
&\quad = \int_0^1 \binom{m}{k} p^k (1-p)^{m-k} (n-1)(1-p)^{n-2}\, dp \\
&\quad = \frac{(n-1)(n+m-k-2)!\,m!}{(m-k)!\,(n+m-1)!} \\
&\quad = \binom{n+m-k-2}{m-k} \Big/ \binom{n+m-1}{m},
\end{aligned} \tag{4}$$

a special case of (1).

To get the cumulative probability, F, that the load on a node is less than or equal to j, we need to sum the probability above for k in the range 0 to j.

$$\begin{aligned}
&F(j) = \Pr(Load \le j) \\
&\quad = \sum_{0\le k\le j} \Pr(Load = k)
\end{aligned} \tag{5}$$

We can compute a simple closed form for the case where $v=1$ and $r=1$. Substituting from (4) into (5) we have:

$$\begin{aligned}
F_{v=1\wedge r=1}(j) &= \sum_{0\le k\le j} \Pr(Load_{v=1\wedge r=1} = k) \\
&= \sum_{0\le k\le j} \binom{n+m-k-2}{m-k} \Big/ \binom{n+m-1}{m} \\
&= 1 - \binom{n+m-j-2}{m-j-1} \Big/ \binom{n+m-1}{m}.
\end{aligned} \tag{6}$$

We are not so lucky in the general case, and the author has found no closed form for (1) substituted into (5).

B. Probability distribution of the Maximum Node Load

Now that we have characterized the distribution of the node load, we would like to use that to determine the distribution of the maximum node load from among the n nodes in the network. This is interesting because it is the node with the largest load that will determine the capacity needed on each node to avoid the most loaded node from failing due to overload and putting the system in danger of collapse.

We would like to tune the node capacity and the distributed hashing algorithm so that with high probability no node is overloaded. One way of looking at this problem is to assume that we have chosen an acceptable probability that there be an overloaded node, α, and to determine for particular settings of m, n, v, and r the object capacity each node should have to keep the overload probability bounded by α.

If we define $M(j) = \Pr(\max_i\{Load_i\} \le j)$, then if the capacity of each node is at least $M^{-1}(1-\alpha)$ then the probability of an overload should be less than α.

If the load on each node were independent, then M would be the cumulative maximum distribution of a set of n independent and identically distributed random variables and it could be calculated as the product of the probabilities that each of the n variables is less than j, that is $M(j) = F(j)^n$. Given this we could then calculate $M^{-1}(1-\alpha)$ as $F^{-1}((1-\alpha)^{1/n})$.

However, the load on each node is not independent. For example, if the load on one node is very high, then all of the other nodes have a lower probability of having a high load. However simulation shows that load acts independently as n grows and hence the formulas above are excellent estimates of the value.

There is an exact formulation for the maximum node load that can be expressed as a recurrence, but this formulation is difficult to work with and unnecessary for larger values of n where this type of algorithm is applied.

C. Standard Deviation of Node Load

While we have no closed form for the inverse of the cumulative maximum load distribution shown above, we can determine closed forms for the standard deviation of the load, which is a good indicator of the inverse maximum. We determine the standard deviation in two ways.

The first way is very straight forward and is based on the formula for the standard deviation of a distribution as

$$\sigma(X) = \sqrt{\sum \Pr(X = i)(i - E(X))^2}. \tag{7}$$

Substituting in $Load_{v=1\wedge r=1}$ from (4) for X in (7) we have:

$$\sigma(Load_{v=1\wedge r=1}) = \sqrt{\sum_{0\le k\le m} \left(k - \frac{m}{n}\right)^2 \binom{n+m-k-2}{m-k} \Big/ \binom{n+m-1}{m}}$$

$$= \sqrt{\frac{(n-1)m(n+m)}{(n+1)n^2}} \,. \tag{8}$$

Unfortunately substituting equation (1) into equation (7) to get a more general version of the formula for standard deviation does not give an expression for which the author can find a closed form.

Not to worry: we can get an expression for the standard deviation when v and r are not equal to one by harking back to our formulation of the probability density of the load on single load element as the convolution of a binomial distribution with a minimum value distribution used in deriving (4) above. We can then use the fact that the variance of the sum of a set of random variables is the sum of the variances of those variables to help us.

First consider the contribution from the binomial distribution: There are m trials, one per object, and while there are nv virtual nodes, this node will get the load if the object hashes to any of the rv virtual nodes that provide load to this node. Thus the probability of the node getting an object is rv/nv. Given that the variance of a binomial distribution with t trials and probability p is $tp(1-p)$, we can determine the contribution to the variance due to the binomial distribution.

Now consider the contribution to the variance from the distribution of the arc length assigned to each node. We have nv virtual nodes being assigned positions on the unit perimeter circle, these positions imply a piece of the arc to be owned by each virtual node. The arc length associated with a node is the sum of the lengths of the rv arcs of the virtual nodes that provide load to this node. As in the derivation of (4) we transform the unit perimeter circle into a unit interval by choosing one of the virtual nodes and unwrapping the circle at that point forming a unit interval with $nv-1$ points. The variance of the minimum of x uniform variables over the unit interval is well known [8] as:

$$\frac{x}{(x+1)^2(x+2)}, \tag{9}$$

and substituting x with $nv-1$ gives us the variance of the proportion of the perimeter that is covered by a node. To get the load it is necessary to scale the proportion of the arc by m, which multiplies the variance by m^2 yielding:

$$\begin{aligned}\sigma(Load) &= \sqrt{m\left(\frac{rv}{nv}\right)\left(1-\frac{rv}{nv}\right)+m^2 rv\frac{(nv-1)}{(nv)^2(nv+1)}} \\ &= \sqrt{\frac{mr}{n}\left(1-\frac{r}{n}\right)+\frac{m^2 r}{n^2 v}\left(1-\frac{2}{nv+1}\right)} \\ &\approx \sqrt{\frac{mr}{n}+\frac{m^2 r}{n^2 v}} \,. \end{aligned} \tag{10}$$

What we would really like to know is not the absolute value of the load, but the ratio between the maximum and expected load. Let's call this ratio the *load factor* of a node. This tells us how many times the ideal amount of resources we need to allocate to each node to avoid overload. Ideally we would like to have a load factor equal to one, but this would mean that the load has been spread perfectly. In a realistic system we would like to bound the maximum load factor to be only a small fraction larger than 1, for example, if the load factor is 1.05 it means that we only need 5% over capacity in the system to prevent overloads. The acceptable percentage of over capacity in the system depends on the application, but one would certainly like this value to be bounded as the size of the network grows.

Scaling the result in (10) by the expected load, mr/n, gives us the standard deviation of the load factor as:

$$\sigma(LoadFactor) \approx \sqrt{\frac{n}{mr}+\frac{1}{rv}} \,. \tag{11}$$

Note that while the standard deviation is related to the maximum value and increases with it, the exact relationship depends on the distribution.

IV. Chord Modifications

We would like to reduce the amount of over capacity needed to prevent overload in the system by reducing the maximum load factor. With that goal in mind, we suggest the following modifications to the Chord algorithm that reduce the standard deviation.

The following modifications are based on Chord's s parameter, which did not appear in any of the formulas above. The number s is the number of consecutive virtual nodes around the circle that each virtual node must monitor. The consecutive virtual nodes must be maintained carefully as, unlike the finger pointers, which are hints used to find data quickly, these nodes are needed to maintain the correct operation of the system. The s parameter has to be large enough so that the probability of a failure causing the loss of any consecutive s nodes around the circle in a short time is negligible, because such a failure would be catastrophic. However making s too large is not good because maintaining the consecutive nodes requires work proportional to sv to be done by each node to monitor its neighbours.

Also note that the replication level r can be at most s because all the nodes maintaining replicas of the same object must closely monitor each other.

A. Spreading the Load

The first observation is that instead of storing the replicas of an object on the first r virtual nodes following the object's key hash, we can store the replicas on any set of r virtual nodes out of the s virtual nodes close to the has value. We can choose which subset of r virtual nodes to choose by rehashing the object's key using a second hash function to obtain one of the $\binom{s}{r}$ different subsets of r nodes chosen from among the s virtual nodes in proximity to the hash node. Using this technique we reduce the variance in load by further smoothing the variations in the arc length.

Because this technique is deterministic we can find the object just as quickly as when using the standard Chord algorithm. When following the fingers in search of an object,

we indicate not only the object and the hash value we are looking for, but the also the offsets from the virtual node holding the hash value of the subset of replicas nodes that we have hashed to.

As in our derivation of (10) we can look separately at the binomial distribution contribution to the variance and the arc length value contribution. The binomial contribution is unchanged as we still have m trials, and the double hashing does not change the probability that a particular node is chosen.

The arc length contribution changes as we now can look at the unit circumference circle split up into nvs components which we add them up vs components at a time to get the variance of the portion of the circle arc assigned to each node. As before, we then scale portion of the circle arc by mr to obtain the number of items landing in the arc, which scales the variance by $(mr)^2$. Giving us:

$$\sigma(SLoad) = \sqrt{m\left(\frac{rv}{nv}\right)\left(1-\frac{rv}{nv}\right)+(mr)^2 vs\frac{(nvs-1)}{(nvs)^2(nvs+1)}}$$

$$= \sqrt{\frac{mr}{n}\left(1-\frac{r}{n}\right)+\frac{m^2r^2}{n^2vs}\left(1-\frac{2}{nvs+1}\right)} \qquad (12)$$

$$\approx \sqrt{\frac{mr}{n}+\frac{m^2r^2}{n^2vs}}.$$

Rescaling (12) by mr/n to get the standard deviation of the load factor rather than the load gives us:

$$\sigma(SLoadFactor) \approx \sqrt{\frac{n}{mr}+\frac{1}{vs}}\,. \qquad (13)$$

The analysis indicates that though we have certainly reduced the variance by using this technique, it is merely equally as effective as increasing the number of virtual nodes and makes no dent in the binomial portion of the variance. If we are going to do significantly better, we need to use a more effective approach as discussed in the next section.

B. Dynamically Balancing the Load

A second observation is that we could do very much better if we could do a bit of dynamic load balancing. One approach to dynamic load balancing is to use the same technique as above, but instead of hashing to a subset of r virtual nodes from among the s surrounding the hashed node, hash to a subset of $r+1$ virtual nodes and place the object on the r nodes with the lowest object loads from among the $r+1$ hash chosen virtual nodes. Because of the dynamic nature of this algorithm, its standard deviation is not easily computed but simulation results, in the next section, show that this reduces the standard deviation dramatically.

Taking this approach requires a bit more work on the part of the object location method, as while our hash function gives us a set of $r+1$ locations for the object, the object will only actually be stored only on r of them. This adds an expected cost of $1/(r+1)$ additional messages, as there is a probability of $1/(r+1)$ that we will choose the node that is not holding a replica of the item we are looking for. From that node, it will take at one additional message to contact one of the r correct nodes. Assuming each virtual node maintains the object load on each of its s nearest neighbours, which it can do as part of the usual ring keep-alive protocol, there is no additional cost on object placement.

We can generalize this algorithm to choose the r lowest load nodes from a chosen from a larger subset of nodes. Consider a secondary hash function that chooses a subset of ss virtual nodes chosen from the s virtual nodes surrounding our hash node, where $r \le ss \le s$. When placing an object we choose the r nodes from that set with the lowest load. There is additional cost in using this technique as when we want to find a replica of an object, we have to search to find one of the r replicas among the set of ss potential sites given by the hash function. Having ss get larger improve the load balancing, but it increases the expected number of wrong nodes we may explore on the way to the right node. This expectation is:

$$\sum_{0\le k\le ss-r} k\binom{ss-r}{k}\Big/\binom{ss}{k} = \frac{(ss+1)(ss-r)}{(r+2)(r+1)}\,. \qquad (14)$$

This can be seen because the probability that we make exactly k wrong moves is the number of ways of choosing k nodes out of the $ss-r$ wrong nodes, divided by the number of ways of choosing k out of ss nodes.

We certainly would like to keep this number small, so we can see that the expected number of extra messages is less than 2 as long as ss is less than $2r$, or if $r=3$, then the expected number of extra messages will be less than 2 as long as $ss \le 7$. Given the cost of finding an object is already O(log n) we could let ss can grow as large as O(r $\log^{1/2} n$) without it affecting the asymptotic runtime.

C. Minimizing Network Overhead

Minimizing the extra capacity needed to avoid overload is important, but equally important is minimizing the cost of accessing an object, mentioned above, and in maintaining such a network. We want a system that simultaneously meets all of these goals. To do this we have to quantify the overhead in running an instance of Chord.

The overhead in Chord comes in three components:

– Maintaining the ring: Each virtual node must monitor each of its s neighbours around the ring to make sure that they are up. This is generally done using 'keep alive' messages sent between all pairs of nodes. Since each node is associated with v virtual nodes, the overhead per node per unit of time is O(vs).

– Maintaining fingers: Each virtual node must maintain the fingers that it uses to quickly find hash locations along the ring. Each virtual node must keep O(log n) fingers, so each node must maintain O(v log n) fingers. Note that the fingers don't have to be maintained as carefully as the ring adjacencies because if they are out of date only the performance is lost.

– Object movement resulting from node failure and repair: Let us assume that each node has a small constant probability λ

of failing or being repaired in some fixed unit of time. Thus the system wide node failure and repair rate is λn and the system wide virtual node failure and repair rate is λnv, as each real node has v virtual nodes. But such an event affects the s virtual nodes next to the virtual nodes that have just been repaired or failed representing giving us a system wide event rate of λnvs. Let's examine the cost of handling one of these events.

The load on node x may be affected in one of three ways by a failure or repair event of a virtual node within distance s: (a) a node that already holds a replica of an object y may tell node x that it should serve object y as node x has just entered the replica set of object y, (b) node x may notice that an object that it is already serving should be replicated to another node that has newly entered the replica set of the object, and finally (c) node x may notice that an object that it is already serving should no longer be served because x is no longer in the replica set of the object. We will count the cost of each event as unit cost, but we need to count the frequencies of the events.

First we can note that the rates of (a) and (b) must be equal, as for a node to be told that it has entered the replica set, another node must have told it. Secondly, since the number of object replicas mr must stay constant in the long term, then (c), the rate of controlled replica removals, must equal the rate of replica additions minus the rate of unplanned replica removals, that is, (c) is (a) – λmr. Thus all we have to do is evaluate the frequency of event (a). The expected number of object replicas assigned to the s virtual nodes around x is mrs/nv and each of these objects replicas is hashed to a pseudo random subset of r out of the s adjacent virtual nodes, thus the probability that one of these objects previous did not previously hash to node x and now does is $\left(\frac{s-r+1}{s}\right)\left(\frac{r}{s}\right)$. Multiplying this out, the rate of object movement for the entire system is:

$$\mathrm{O}\left(\lambda nvs\left(\frac{mrs}{nv}\right)\left(\frac{s-r+1}{s}\right)\left(\frac{r}{s}\right)\right)=\mathrm{O}\left(\lambda mr^2(s-r+1)\right). \qquad (15)$$

Finally, adding up all the overhead costs, noting that r and λ are constants and that dividing the system object movement rate by n to get the per node rate, we can determine that the overall per node maintenance cost per unit time is:

$$\mathrm{O}(vs + v \log n + ms/n). \qquad (16)$$

If we assume that m/n is a constant, this formula suggests that the best cost-performance will be when v and ss are set as small constants, and when s is no more than O(log n). Having a logarithmic spread rather than a logarithmic number of virtual nodes, as has been previously proposed, results in better load balancing while reducing the cost of running the system by allowing a smaller number of virtual nodes per real node.

V. Simulation Results

The simulation results reported here were computed using uniform pseudo random variables instead of actual node IDs, object keys, and hash functions. We assume that the hash functions evaluated on the actual keys and node IDs will approximate the behaviour seen in this simulation.

The first step in the simulation is to generate the mappings to simulate a hash function that picks a subset of ss nodes chosen out of s nodes. We then choose a set of v pseudo random numbers to simulate the hash results for each of the n nodes. These hash results are linked so that the load from each virtual node can be combined. We then generate a primary hash key for each of the m objects and a secondary subset hash. The primary hash is used to find the virtual node associated with the hash value and the secondary hash is used to determine the subset of the ss out of s contiguous virtual nodes around the hashed virtual node to examine. The object load is then added to the r nodes with the lowest load from among the nodes corresponding with the ss virtual nodes chosen above.

When all the objects are placed we then examine the load factor of each of the n nodes to determine the mean, standard deviation and maximum loads. This is then repeated for 100 trials. We report here the standard deviation of the node load factor and the capacity load factor needed on each node in order to keep the probability of overload on any node less than α, that is $M^{-1}(1-\alpha)$ scaled by the expected load, mr/n. We label these quantities σ(LF) and max(LF) in the tables below. All values reported are experimental; the correspondence with the formulas developed in the previous sections is very good.

Because of the large number of variables, n, m, v, r, s, ss, and α, we choose only a small subset of the results to vary in each table. More results can be examined on my web site [9] or with the simulation program, which we have made available so that you can generate your own values. In all of our reported results we set n to 1000, and r to 3. These are reasonable but arbitrary values. We also set α to 0.1 in most of our reported results, a somewhat large value, meaning that 10% of the time the system may fail due to overloading. The effect of changing α is shown in Table VI.

TABLE I: STANDARD CHORD LOAD

$v \setminus m$		10000	100,000	1,000,000
1	σ(LF)	0.61	0.58	0.58
	max(LF)	4.63	4.42	4.37
2	σ(LF)	0.45	0.41	0.41
	max(LF)	3.43	3.36	3.16
13	σ(LF)	0.24	0.17	0.16
	max(LF)	2.07	1.72	1.71
18	σ(LF)	0.23	0.15	0.14
	max(LF)	2.03	1.62	1.58
∞	σ(LF)	0.18	0.058	0.018
	max(LF)	1.73	1.22	1.07

First, for comparison purposes we set $s = ss = r$. This gives us the standard Chord algorithm with the replicas assigned to the r virtual nodes adjacent to the hash value. We report this for v set to 1, 2, 13, and 18 and m set to 10,000, 100,000, and

1,000,000, that is for 10, 100, and 1000 objects per node in Table I. The final row gives the results for v approaching infinity, simulated by using identical length arcs for each virtual node.

As shown, increasing v can be effective at reducing the load factor, however for a given value of m the effect of v is limited, as increasing v does not affect the binomial contribution to the variance and each increase of v has less effect on the load.

TABLE II: EXTENDED SPREAD LOAD

$v \setminus s$		3	8	13	18
1	σ(LF)	0.58	0.36	0.28	0.24
	max(LF)	4.42	2.82	2.32	2.07
2	σ(LF)	0.41	0.26	0.21	0.18
	max(LF)	3.36	2.22	1.93	1.80
3	σ(LF)	0.34	0.21	0.17	0.15
	max(LF)	2.76	1.95	1.76	1.67
13	σ(LF)	0.17	0.11	0.096	0.089
	max(LF)	1.72	1.48	1.40	1.36
18	σ(LF)	0.15	0.10	0.088	0.081
	max(LF)	1.62	1.42	1.36	1.33

In Table II we examine using larger spreads without dynamic load balancing. We fix m at 100,000, that is, 100 objects per node, and since we are not using dynamic load balancing we set $ss = r$ and look at s = 3, 13, and 18 and v = 1, 2, 3, 13, and 18 in Table II. Since r is set to 3, we cannot consider values of s of less than 3.

We note that the result is symmetric in s and v as expected. By adjusting s and v separately we can reduce the maximum load more efficiently than leaving s fixed at 3 and just adjusting v. For example, without increasing the per node overhead appreciably we can decrease wastage from 62% to 33% by increasing s from 3 to 18. However, as with standard Chord, no matter how high we set s and v, we are still limited by the binomial component to a CLF of 1.22.

Now we examine the load when using dynamic load balancing. We look separately at v set to 1, 2 and 3, along with s set to 8, 13 and 18 and at the subset size, ss, varying from 3 (no load balancing) to 6 (choosing the least loaded 3 out of 6 nodes) in Tables III, IV and V. Note that with dynamic load balancing enabled, the formulas developed in the previous sections no longer apply.

Increasing v from one to two makes a dramatic improvement in the standard deviation, reducing it by a factor of four, while increasing v from two to three reduces the standard deviation by only 30%. We conjecture that the use of virtual nodes allows nodes to balance load with nodes quite far apart along the virtual node circle, but once it has been done, playing the same trick a second time makes little profit.

Increasing the size of the load balance subset, ss, makes a significant improvement to the standard deviation: each increment causes a four-time drop in the standard deviation. Similarly, increasing the spread, s, also makes a significant improvement, each doubling of the spread causing a halving of the standard deviation.

However, once the standard deviation is low enough so that the maximum load is close enough to optimal, it makes little sense to pay more for even better load balancing.

TABLE III: DYNAMIC LOAD, $v = 1$

$s \setminus ss$		3	4	5	6
8	σ(LF)	0.36	0.26	0.22	0.19
	max(LF)	2.82	2.05	1.92	1.70
13	σ(LF)	0.28	0.19	0.15	0.13
	max(LF)	2.32	1.63	1.51	1.46
18	σ(LF)	0.24	0.15	0.12	0.11
	max(LF)	2.07	1.52	1.38	1.34

TABLE IV: DYNAMIC LOAD, $v = 2$

$s \setminus ss$		3	4	5	6
8	σ(LF)	0.26	0.082	0.033	0.015
	max(LF)	2.00	1.13	1.044	1.023
13	σ(LF)	0.21	0.049	0.015	0.0058
	max(LF)	1.80	1.043	1.017	1.010
18	σ(LF)	0.18	0.034	0.0089	0.0039
	max(LF)	1.60	1.030	1.013	1.010

TABLE V: DYNAMIC LOAD, $v = 3$

$s \setminus ss$		3	4	5	6
8	σ(LF)	0.21	0.051	0.016	0.0066
	max(LF)	1.99	1.037	1.013	1.010
13	σ(LF)	0.17	0.030	0.0073	0.0036
	max(LF)	1.78	1.023	1.010	1.0067
18	σ(LF)	0.15	0.020	0.0052	0.0031
	max(LF)	1.65	1.020	1.010	1.0067

Therefore we suggest that we start with setting ss to r +1 and v to 2 and s to log n. Making these settings we get minimal wastage of capacity, about 5% as we see above, and minimal overhead of O(log n + mr/n) per node. The use of more than two virtual nodes per real node can then be reserved for very high capacity nodes that use multiple virtual nodes; not to smooth the load, but only to increase a particular node's share of the already smoothed load.

Finally we look at the sensitivity of these results on m and α. We again set m to 10000, 100,000 and 1,000,000 and we look at α set to 0.5, 0.1, and 0.01. In this table we fix $s = 13$, $ss = 4$ and $v = 2$. To get good results for such small values of α we also increased the trials from 100 to 1000. For each experiment we report the capacity load factor needed on each node to ensure a failure probability no greater than α.

TABLE VI: DYNAMIC LOAD, v=2, s=13, ss=4

$\alpha \setminus m$		10,000	100,000	1,000,000
0.5	max(LF)	1.20	1.036	1.014
0.1	max(LF)	1.23	1.043	1.017
0.01	max(LF)	1.30	1.060	1.025

These results show how dynamic loading breaks through the binomial barrier. It also shows that with tens of objects per node that getting low maximums is difficult. Finally it shows that using smaller values of α does not cause significantly greater expense.

VI. Conclusion

In this paper we have accomplished three things:

– We have analyzed the variation in load that the Chord algorithm puts on its participant nodes and discussed the effect that this has on the ability of the Chord algorithm to use all its resources.

– We have introduced modifications to the Chord algorithm that allow for increased load spreading and for dynamic load balancing while adding much less overhead than increasing the number of virtual nodes per real node.

– Finally we have run simulations of the new algorithm and measured the increase in the utilization of the resources in the network.

Future work will include an implementation of this algorithm that can run on real and simulated network systems. This will answer definitively whether the modifications to consistent hashing presented here are practical for a real system and will yield the promised payoffs in reduced overhead and resource utilization.

References

[1] D. Karger, E. Lehman, T. Leighton, M. Levine, D. Lewin, and R. Panigrahy. Relieving hot spots on the World Wide Web. In Proceedings of the 29th Annual ACM Symposium on the Theory of Computing, pages 654--663, May 1997.

[2] David Lomet Rick Anderson, T. K. Rengarajan, Peter Spiro, How the Rdb/VMS Data Sharing System Became Fast. Digital Equipement Corporation, Tech Report CRL 92/4. May 29, 1992. www.hpl.hp.com/techreports/Compaq-DEC/CRL-92-4.pdf.

[3] Ion Stoica, Robert Morris, David Liben-Nowell, David R. Karger, M. Frans Kaashoek, Frank Dabek, Hari Balakrishnan, Chord: A Scalable Peer-to-peer Lookup Protocol for Internet Applications. IEEE/ACM Transactions on Networking, 11(1), February 2003, pp. 17 – 32.

[4] A. Rowstron and P. Druschel, "Pastry: Scalable, distributed object location and routing for large-scale peer-to-peer systems". IFIP/ACM International Conference on Distributed Systems Platforms (Middleware), Heidelberg, Germany, pages 329-350, November, 2001.

[5] Hung-Chang Hsiao, Chung-Ta King: Tornado: A Capability-Aware Peer-to-Peer Storage Network. IPDPS 2003: 72.

[6] Ryan Huebsch, Joseph M. Hellerstein, Nick Lanham, Boon Thau Loo, Scott Shenker, Ion Stoica, Querying the Internet with PIER. VLDB, 2003.

[7] Frank Dabek, M. Frans Kaashoek, David Karger, Robert Morris, and Ion Stoica, Wide-area cooperative storage with CFS, ACM SOSP 2001, Banff, October 2001.

[8] Eric W. Weisstein. “Probability and Statistics.” From MathWorld – A Wolfram Web Resource. http://mathworld.wolfram.com/topics/ProbabilityandStatistics.html.

[9] Garret Swart. Back up material for “Spreading the Load using Consistent Hashing.” http://www.cs.ucc.ie/~gs1/SpreadLoad.